BÁO CÁO PROJECT LEVEL_2

A. TỔNG QUAN (OVERVIEW)

1. Thông tin cá nhân:

- Họ tên: Nguyễn Thảo Ly
- MSSV: 25022931

2. Tổng quan thiết kế luồng thực thi:

- Chương trình được thực hiện theo luồng tuần tự:
  init() → startGame() → playGame() → endGame() → close()

- Engine đóng vai trò trung tâm điều phối, làm việc hoàn toàn thông qua interface trừu tượng là I_renderer và I_interaction. Không phục thuộc vào bất kỳ cài đặt cụ thể nào. Nhờ đó chương trình có thể chạy trên cả 2 giao diện Terminal và SDL mà không cần thay đổi logic lõi của engine.

- Khi khởi động, chương trình đọc tham số dòng lệnh qua parseArgs() để xác định chế độ chạy. Nếu có flag -g (./game.exe -g) thì khởi tạo SDLRenderer và SDLInteraction, ngược lại nếu không có flag -g thì chạy TerminalRenderer và TerminalInteraction.

- Sau đó engine gọi startGame() để thu thập thông tin từ người dùng (kích thước bản cờ, chế độ chơi,...) khởi tạo bàn cờ rồi bước vào vòng lặp chính playGame(). Mỗi lượt, Engine hiển thị bàn cờ nhận nước đi từ người chơi hoặc bot, kiểm tra thắng thua rồi chuyển lượt. Khi game kết thúc, endGame() hiển thị kết quả và close() giải phóng tài nguyên.

3. Giao diện đồ họa:

- Giao diện đồ họa được xây dựng bằng thư viện SDL2, sử dụng SDL_ttf để hiện thị chữ và SDL_image để load ảnh nền. Bàn cờ được vẽ bằng các hình chữ nhật (SDL_RenderDrawRect), ký hiệu X và O được render bằng font chữ với màu sắc phân biệt. Giao diện hỗ trợ tương tác bằng cả chuột (click vào ô để đánh) và bàn phím (nhập số để chọn size, goal, chế độ chơi, bot level).

B. CHI TIẾT CÀI ĐẶT (IMPLEMENTATION DETAIL)

1. Tư duy thiết kế:

- Luồng thực thi của Engine giữ nguyên cấu trúc tuần tự từ Level1: init() → startGame() → playGame() → endGame() → close()

- Điểm khác biệt so với level 1 là các thao tác hiển thị và nhận input không còn gọi trực tiếp cout/cin mà đều đi qua hai interface I_Renderer và I_Interaction. Engine không biết và không cần biết đang chạy Terminal hay SDL, nó chỉ gọi iRenderer → displayBoard() hay iInteraction → getPlayerMove() là xong.

- Ưu điểm của thiết kế này là dễ mở rộng, muốn thêm giao diện mới chỉ cần tạo class mới implement interface, không cần sửa Engine.

- Nhược điểm là luồng tuần tự không phù hợp hoàn toàn với SDL vì SDL hoạt động theo event-driven, đôi khi cửa sổ bị "Not Responding" khi chờ input quá lâu.

2. Trả lời câu hỏi:

2.1. Lập trình hướng đối tượng:

- Tính đóng gói: Mỗi class che giấu dữ liệu nội bộ qua private. Ví dụ SDLRenderer có window, renderer, font là private, người dùng chỉ gọi displayBoard() mà không biết bên trong dùng SDL như nào.

- Tính kế thừa: Hệ thống bot được tổ chức theo chuỗi kế thừa Bot → BotLevel1 → BotLevel2 → BotLevel3. Botlevel2 kế thừa random_pick() từ BotLevel1, BotLevel3 kế thừa simple_heuristic() từ BotLevel2 và mở rộng minimax().

- Tính trừu tượng: I_Renderer và I_Interaction là các interface trừu tượng định nghĩa hành vi mà không cài đặt chi tiết. Engine chỉ làm việc với interface không biết bên trong là Terminal hay là SDL.

- Tính đa hình: Engine giữ con trỏ I_Renderer*và I_Interaction*. Khi gọi iRenderer → displayBoard(), runtime tự quyết định gọi TerminalRenderer::displayBoard() hay SDLRenderer::displayBoard() tùy object được gắn vào.

2.2. Thiết kế cân bằng hai giao diện:

- Thiết kế có cân bằng giữa Terminal và SDL nhờ interface chung I_Renderer và I_Interaction. Cả hai giao diện đều implement đầy đủ các hàm trong interface nên Engine hoạt động thống nhất. Phần setup (chọn size, goal, mode, bot level) ở SDL vẫn dùng bàn phím để nhập số, tương tự Terminal giúp luồng thực thi của Engine không cần thay đổi. Phần chơi game thì SDL dùng chuột để click vào ô, còn Terminal dùng bàn phím nhập tọa độ.

2.3. Đánh giá mức độ tuân thủ SOLID:

- S – Single Responsibility: Mỗi class có một trách nhiệm rõ ràng: Engine điều phối, Logic xử lý luật chơi, TerminalRenderer hiển thị terminal, SDLRenderer hiển thị đồ họa, Logger ghi Log.
  → Tuân thủ tốt.

- O – Open/Closed: Muốn thêm giao diện mới chỉ cần tạo class mới implement I_Renderer và I_Interaction mà không cần sửa Engine.
  → Tuân thủ tốt.

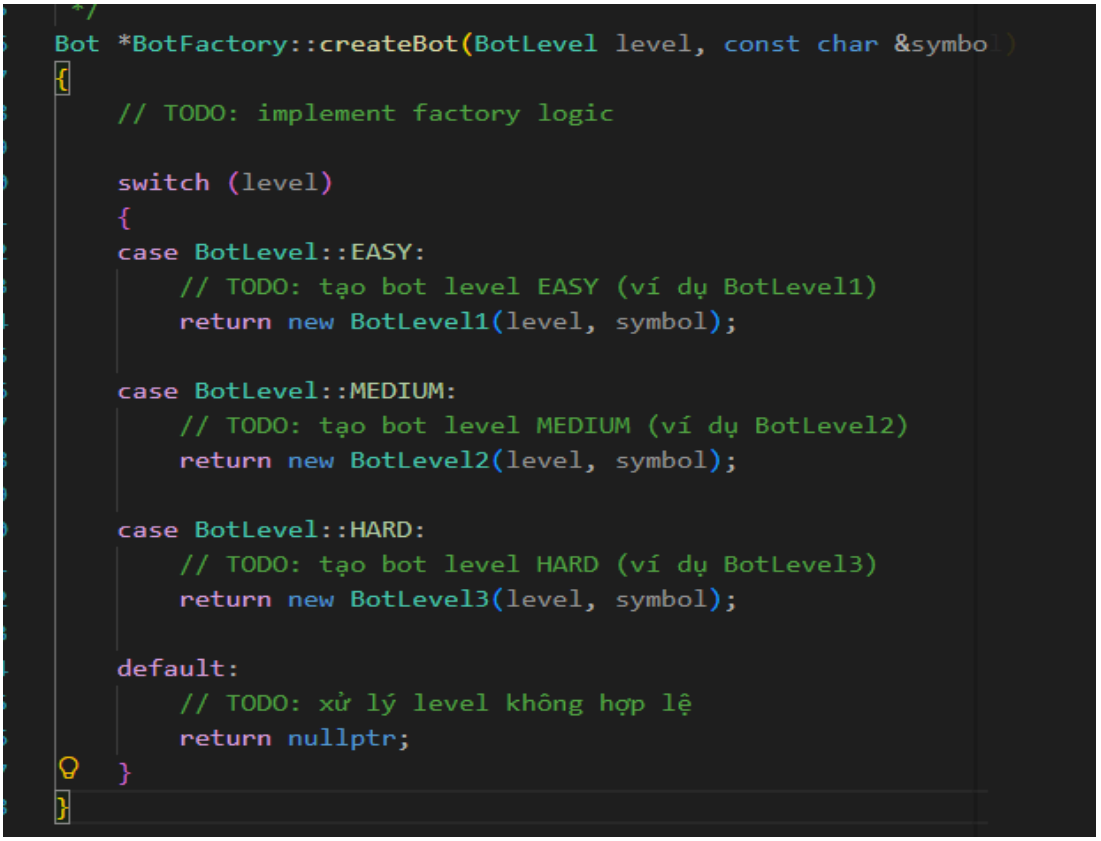
- L – Liskov Substitution: TerminalRenderer và SDLRenderer có thể thay thế nhau thông qua con trỏ I_Renderer* mà không làm sai hành vi Engine.
  → Tuân thủ tốt.

- I – Interface Segregation: I_Renderer và I_Interaction được tách thành hai interface riêng biệt, không ép các class impolement những hàm không cần thiết.
  → Tuân thủ tốt.

- D – Dependency Inversion: Engine phụ thuộc vào I_Renderer và I_Interaction là abstraction, không phụ thuộc vào TerminalRenderer hay SDLRenderer cụ thể.
  → Tuân thủ tốt.

2.4. Sử dụng mẫu thiết kế:

- Chương trình sử dụng Factory Pattern trong BotFactory::createBot() — nhận vào BotLevel và symbol, trả về con trỏ Bot* tương ứng với level đó. Engine không cần biết đang tạo BotLevel1, BotLevel2 hay BotLevel3 — chỉ cần gọi BotFactory::createBot() là có bot phù hợp. Điều này giúp dễ thêm bot mới mà không cần sửa Engine.
- minh chứng:



```
 */
Bot *BotFactory::createBot(BotLevel level, const char &symbo
{
    // TODO: implement factory logic

    switch (level)
    {
    case BotLevel::EASY:
        // TODO: tạo bot level EASY (ví dụ BotLevel1)
        return new BotLevel1(level, symbol);

    case BotLevel::MEDIUM:
        // TODO: tạo bot level MEDIUM (ví dụ BotLevel2)
        return new BotLevel2(level, symbol);

    case BotLevel::HARD:
        // TODO: tạo bot level HARD (ví dụ BotLevel3)
        return new BotLevel3(level, symbol);

    default:
        // TODO: xử lý level không hợp lệ
        return nullptr;
    }
}
```

3. Hình thức hiển thị:

- giao diện GUI 2D, trực quan bằng cách phối hợp bộ ba thư viện cốt lõi: SDL2 (quản lý cửa sổ và đồ họa), SDL_image (xử lý hình ảnh) và SDL_ttf (xử lý phông chữ).

- Font & Background: trong hàm init(), hệ thống nạp phông chữ VHVOGU_0.TTF với hai kích cỡ khác nhau: cỡ 32 dùng cho các văn bản thường và cỡ 60 dùng cho tiêu đề chính. Nạp một hình ảnh nền meo.png trên assets được tải lên thông qua IMG_Load.

- Bàn cờ: Kích thước bàn cờ không cố định mà được tính toán tự động trong hàm displayBoard(). Dựa vào kích thước màn hình

screenWidth, screenHeight và khoảng cách lề boardPadding, hệ thống tự động tính ra vùng hiển thị lớn nhất có thể của bàn cờ, từ đó chia đều cho số lượng ô cờ để có được kích thước chuẩn của từng ô vuông đơn lẻ (cellsize = boardsize / gamesize).

- Lý do lựa chọn hình thức:
  so với terminal chỉ hiện thị bằng văn bản, ký tự thuần túy (dùng cout để in ‘|’ , ’-’, ’X, ’O’,...) thì GUI có thể hiển thị bằng hình ảnh, có cửa sổ game riêng, màu sắc, font chữ nghệ thuật. Terminal tuy dễ viết code logic, không tốn tài nguyên máy tính nhưng muốn trông đẹp mắt, chuyên nghiệp có thể tương tác bằng chuột thì GUI là lựa chọn tốt hơn. Nói ngắn gọn thì sử dụng giao diện đồ họa mang lại trải nghiệm tốt hơn vì giao diện dễ nhìn, người dùng dễ quan sát.

- Cơ sở cài đặt:
  Vẽ bảng và ký hiệu: Hàm displayBoard() quét qua ma trận hai chiều board. Với mỗi ô cờ, nó gọi hàm drawRect để vẽ một khung viền hình vuông màu xám. Nếu ô chứa ký tự 'X', hệ thống render chữ "X" màu đỏ; nếu là 'O', hệ thống render chữ "O" màu xanh lá, cả hai đều được dịch tâm bằng công thức hình học (x/y + cellsize / 2 - 19) để đảm bảo quân cờ luôn nằm chính giữa ô.

```cpp
void SDLRenderer::displayBoard(const char board[][BOARD_N_MAX], const int size)
{
    // TODO: Render board
    gamesize = size;
    int boardsize = std::min(screenWidth, screenHeight) - boardPadding * 2;
    int cellsize = boardsize / gamesize;

    drawRect((screenWidth - boardsize) / 2 - 10, boardPadding - 10, boardsize + 20, boardsize + 20, {0, 0, 0, 120}, true);

    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j < size; j++)
        {
            int x = (screenWidth - boardsize) / 2 + j * cellsize;
            int y = boardPadding + i * cellsize;
            drawRect(x, y, cellsize, cellsize, {200, 200, 200, 255}, false);
            if (board[i][j] == 'X')
            {
                renderTitleText("X", x + cellsize / 2 - 19, y + cellsize / 2 - 18, {255, 50, 50, 255});
            }
            else if (board[i][j] == 'O')
            {
                renderTitleText("O", x + cellsize / 2 - 19, y + cellsize / 2 - 18, {0, 255, 0, 255});
            }
        }
    }
    if (lastmoveRow >= 0 && lastmoveCol >= 0)
    {
        int x = (screenWidth - boardsize) / 2 + lastmoveCol * cellsize;
        int y = boardPadding + lastmoveRow * cellsize;
        drawRect(x, y, cellsize, cellsize, {255, 255, 0, 80}, true);
    }

    renderPresent();
}
```

- Highlight move: Để người chơi dễ theo dõi, chương trình lưu lại tọa độ nước đi cuối cùng qua hàm showMove(). Tại displayBoard(), một hình chữ nhật màu vàng có độ trong suốt cao sẽ được phủ lên ô cờ này để làm nổi bật.

```cpp
void SDLRenderer::showMove(const int row, const int col)
{
    // TODO: Highlight move
    lastmoveRow = row;
    lastmoveCol = col;
}
```

- Hiển thị Menu và trạng thái bàn cờ: Hàm showSelectMenu() sử dụng switch-case để vẽ các màn hình nhập vào Size, Goal, Chọn Mode, Chọn Level Bot. Hộp thoại thông báo lỗi showInvalidSelect hay thông báo thành công showValidSelect đều được vẽ bằng các dải màu có độ trong suốt alpha = 200 như màu đỏ hoặc màu xanh phủ lên trên, tạo hiệu ứng UI hiện đại.

- WinLines: Khi trò chơi kết thúc và tìm ra người thắng, hàm showResult() sẽ nhận vào một danh sách các tọa độ nằm trong cấu trúc winLine. Chương trình lập tức duyệt qua các ô này và tô đè một lớp màu vàng Alpha = 120 để highlight rõ ràng đường cờ đã giành chiến thắng trước khi làm mờ toàn bộ màn hình để hiện thông báo kết quả cuối cùng.

- Hiệu năng hiển thị đồ họa:

```
$ ./game.exe -g
[LOG TOTAL] Chay duoc: 1 frames | Thoi gian render TB: 19.944 ms | FPS trung binh: 50.1404
[LOG TOTAL] Chay duoc: 2 frames | Thoi gian render TB: 11.9176 ms | FPS trung binh: 83.9095
[LOG TOTAL] Chay duoc: 3 frames | Thoi gian render TB: 570.09 ms | FPS trung binh: 1.75411
[LOG TOTAL] Chay duoc: 4 frames | Thoi gian render TB: 428.059 ms | FPS trung binh: 2.33613
[LOG TOTAL] Chay duoc: 5 frames | Thoi gian render TB: 768.434 ms | FPS trung binh: 1.30135
[LOG TOTAL] Chay duoc: 6 frames | Thoi gian render TB: 640.672 ms | FPS trung binh: 1.56086
[LOG TOTAL] Chay duoc: 7 frames | Thoi gian render TB: 781.538 ms | FPS trung binh: 1.27953
[LOG TOTAL] Chay duoc: 8 frames | Thoi gian render TB: 684.141 ms | FPS trung binh: 1.46169
[LOG TOTAL] Chay duoc: 9 frames | Thoi gian render TB: 647.454 ms | FPS trung binh: 1.54451
[LOG TOTAL] Chay duoc: 10 frames | Thoi gian render TB: 582.833 ms | FPS trung binh: 1.71576
[LOG TOTAL] Chay duoc: 11 frames | Thoi gian render TB: 566.005 ms | FPS trung binh: 1.76677
[LOG TOTAL] Chay duoc: 12 frames | Thoi gian render TB: 518.958 ms | FPS trung binh: 1.92694
[LOG TOTAL] Chay duoc: 13 frames | Thoi gian render TB: 479.421 ms | FPS trung binh: 2.08585
[LOG TOTAL] Chay duoc: 14 frames | Thoi gian render TB: 445.311 ms | FPS trung binh: 2.24562
[LOG TOTAL] Chay duoc: 15 frames | Thoi gian render TB: 415.952 ms | FPS trung binh: 2.40413
[LOG TOTAL] Chay duoc: 16 frames | Thoi gian render TB: 390.092 ms | FPS trung binh: 2.5635
[LOG TOTAL] Chay duoc: 17 frames | Thoi gian render TB: 367.404 ms | FPS trung binh: 2.7218
[LOG TOTAL] Chay duoc: 18 frames | Thoi gian render TB: 347.044 ms | FPS trung binh: 2.88148
[LOG TOTAL] Chay duoc: 19 frames | Thoi gian render TB: 328.94 ms | FPS trung binh: 3.04007
[LOG TOTAL] Chay duoc: 20 frames | Thoi gian render TB: 312.56 ms | FPS trung binh: 3.19938
[LOG TOTAL] Chay duoc: 21 frames | Thoi gian render TB: 297.853 ms | FPS trung binh: 3.35737
[LOG TOTAL] Chay duoc: 22 frames | Thoi gian render TB: 284.363 ms | FPS trung binh: 3.51663
[LOG TOTAL] Chay duoc: 23 frames | Thoi gian render TB: 272.161 ms | FPS trung binh: 3.67429
[LOG TOTAL] Chay duoc: 24 frames | Thoi gian render TB: 260.872 ms | FPS trung binh: 3.8333
[LOG TOTAL] Chay duoc: 25 frames | Thoi gian render TB: 250.594 ms | FPS trung binh: 3.99051
[LOG TOTAL] Chay duoc: 26 frames | Thoi gian render TB: 240.999 ms | FPS trung binh: 4.1494
```

4. Tương tác:

- Cách cài đặt:
  Phần xử lý tương tác của trò chơi được cài đặt SDLInteraction. Chương trình áp dụng Event-driven dựa trên vòng lặp kiểm tra sự kiện SDL_WaitEvent(). Hàm này có cơ chế chặn, nghĩa là nó sẽ tạm dừng luồng thực thi để chờ cho đến khi nhận được một tín hiệu tác động từ phía người dùng mới tiếp tục xử lý, giúp tối ưu hóa hiệu năng CPU khi người chơi đang suy nghĩ.

- Chương trình hỗ trợ những tương tác:
  Tương tác bằng bàn phím (SDL_KEYDOWN): được dùng chủ yếu tại các màn hình Menu thiết lập ban đầu. Người dùng ấn các phím số từ 0 đến 9 để nhập kích thước bàn cờ (selectSize) và số ô chiến thắng (selectGoal); ấn các phím chức năng 1, 2, 3 để lựa chọn

nhanh chế độ chơi (selectGameMode) và cấp độ thông minh của Bot (selectBotLevel). Hệ thống cũng hỗ trợ phím BACKSPACE để xóa ký tự nhập sai và phím RETURN để xác nhận dữ liệu.

- Tương tác bằng chuột (SDL_MOUSEBUTTONDOWN): Được áp dụng trực tiếp trong trận đấu tại hàm getPlayerMove(). Khi người chơi click chuột lên màn hình, chương trình sẽ bắt tọa độ pixel thực tế, sau đó hệ thống sử dụng thuật toán quy đổi hình học dựa trên kích thước ô cờ cellsize và khoảng lề boardPadding để tính toán chính xác chỉ số hàng row và cột col của ma trận bàn cờ.

```
*row = (mouseY - boardPadding) / cellsize;
*col = (mouseX - (screenWidth - boardsize) / 2) / cellsize;
```

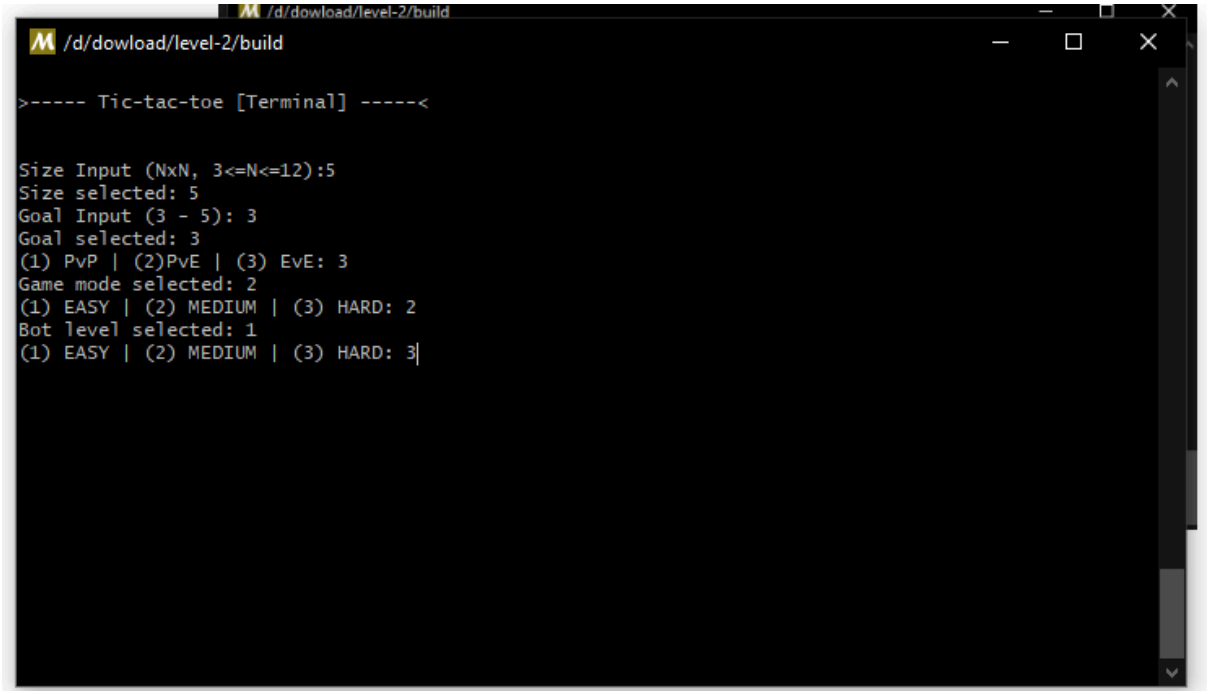
- Hàm waitForQuit() liên tục lắng nghe hành động bấm nút X để đóng cửa sổ người dùng, lập tức ném ra một QuitException nhằm giải phóng tài nguyên và thoát game an toàn.

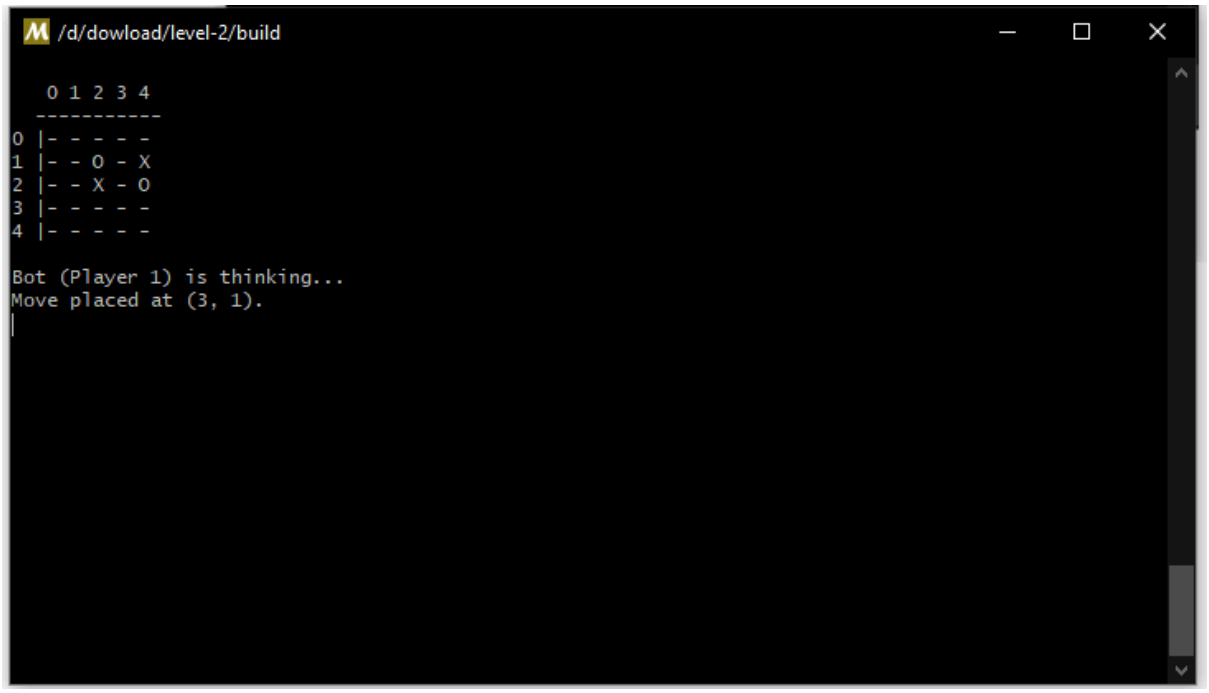
- Khó khăn lớn nhất:
  SDL_WaitEvent bắt buộc phải có sự kiện chuột hoặc phím thì mới chạy tiếp, nên khi chuyển sang lượt của Bot, game không thể nhận được sự kiện click từ máy tính. Ngược lại, nếu sử dụng thuật toán đệ quy sâu (như Minimax của Bot level 3) trên cùng một luồng với giao diện, toàn bộ cửa sổ GUI sẽ bị rơi vào trạng thái *Not Responding* bất chợt.

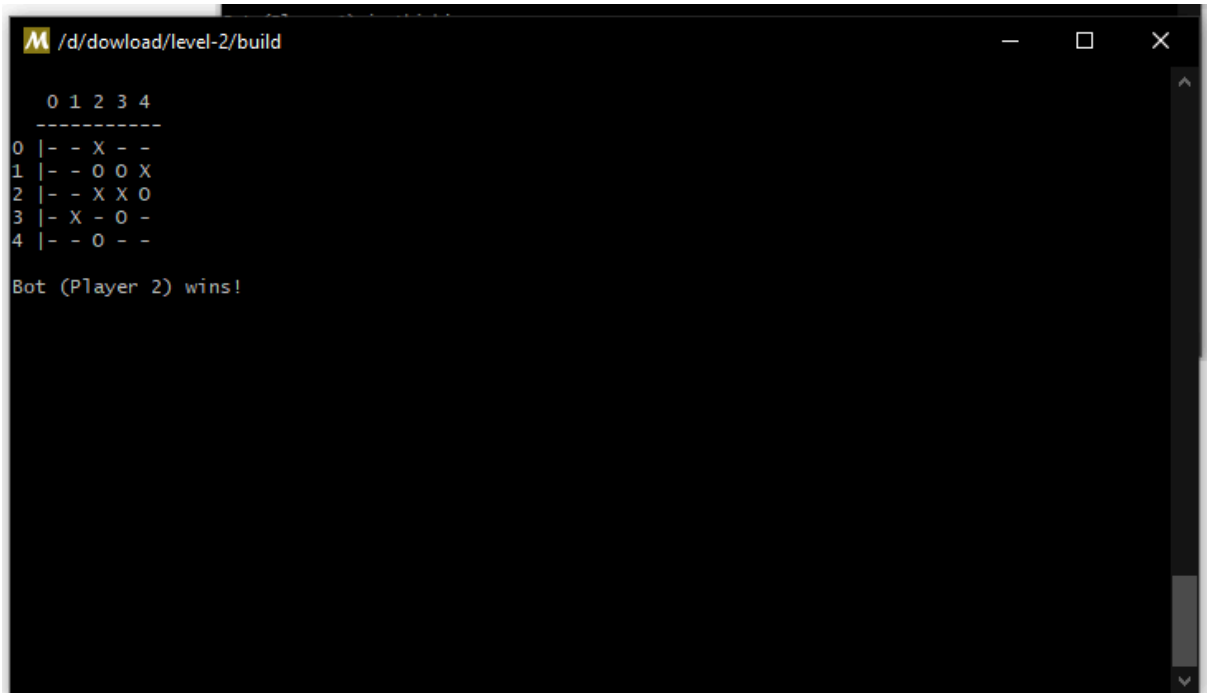
C. KẾT QUẢ THỰC NGHIỆM (EXPERIMENT RESULTS)

***) Ảnh chụp màn hình:**

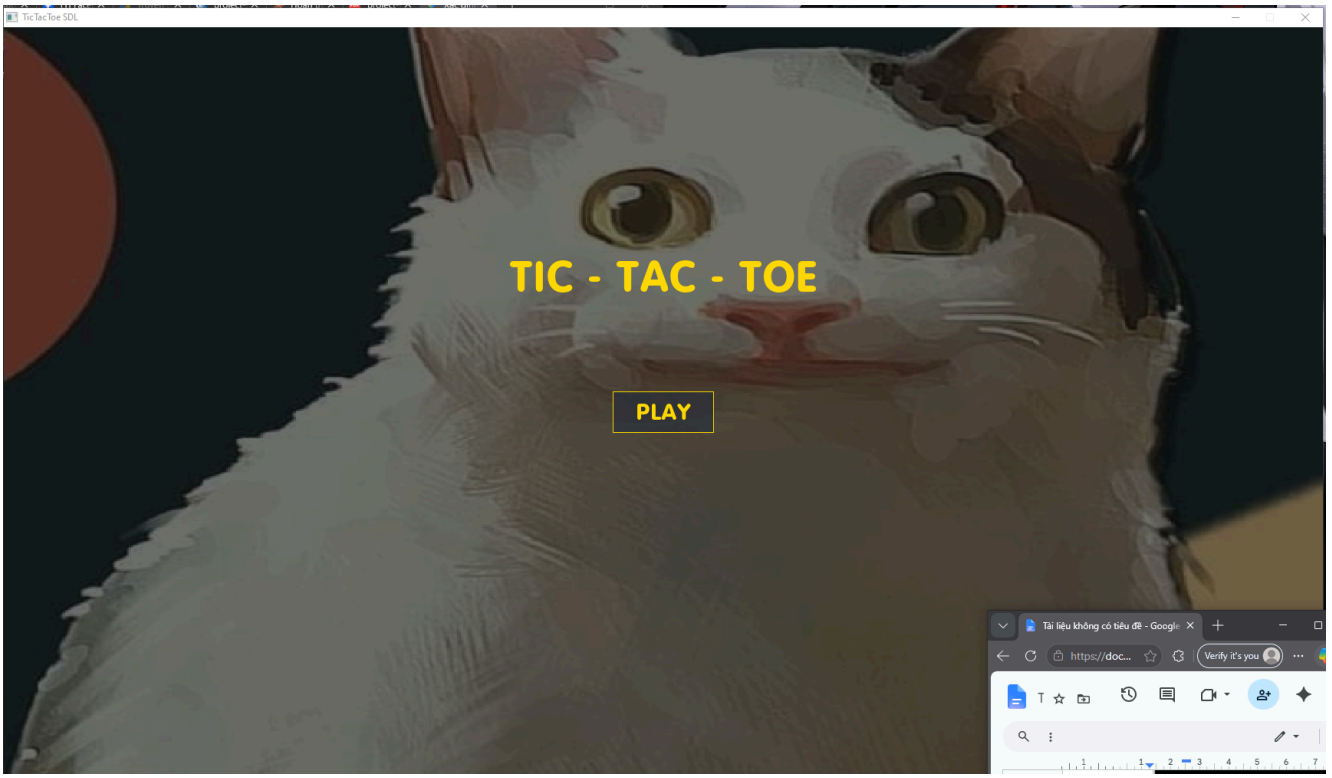
- Terminal: (Menu, displayboard, result)

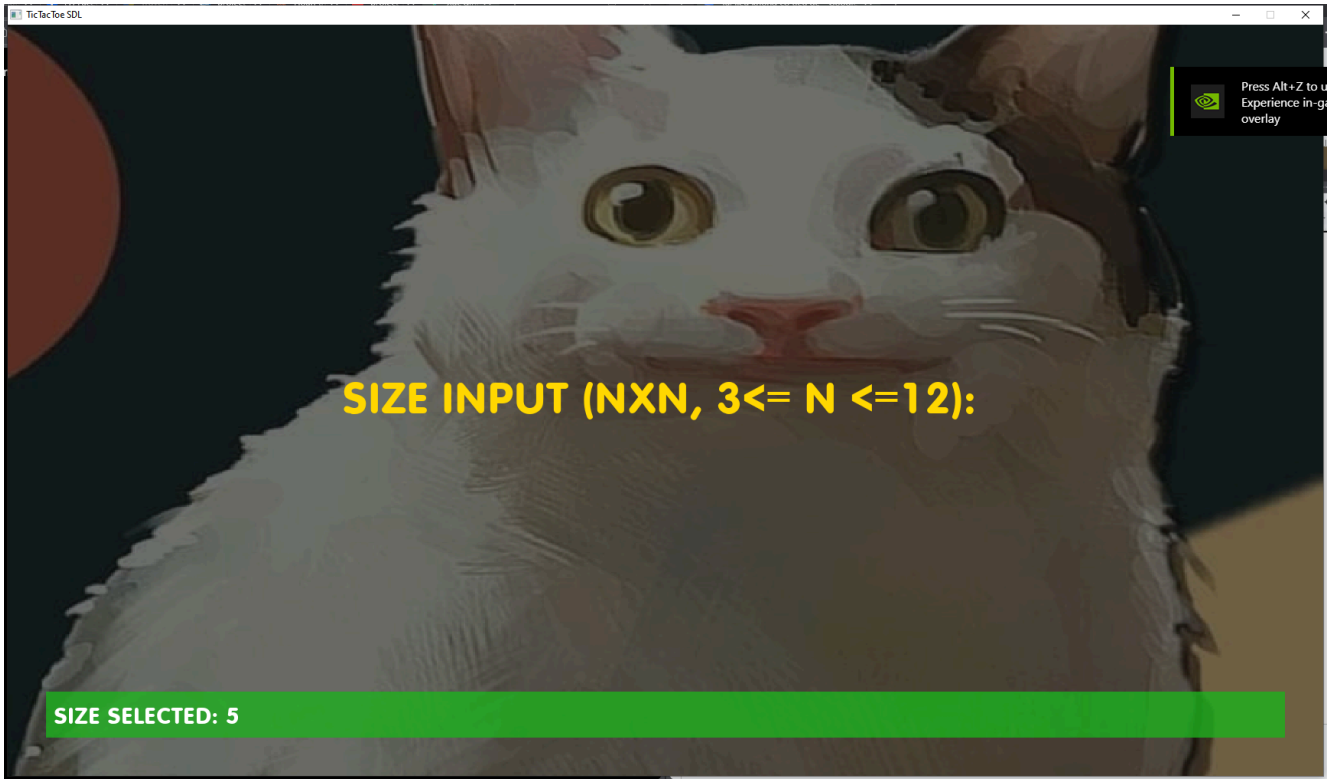

/d/dowload/level-2/build
>----- Tic-tac-toe [Terminal] -----<
Size Input (NxN, 3<=N<=12):5
Size selected: 5
Goal Input (3 - 5): 3
Goal selected: 3
(1) PvP | (2)PvE | (3) EvE: 3
Game mode selected: 2
(1) EASY | (2) MEDIUM | (3) HARD: 2
Bot level selected: 1
(1) EASY | (2) MEDIUM | (3) HARD: 3

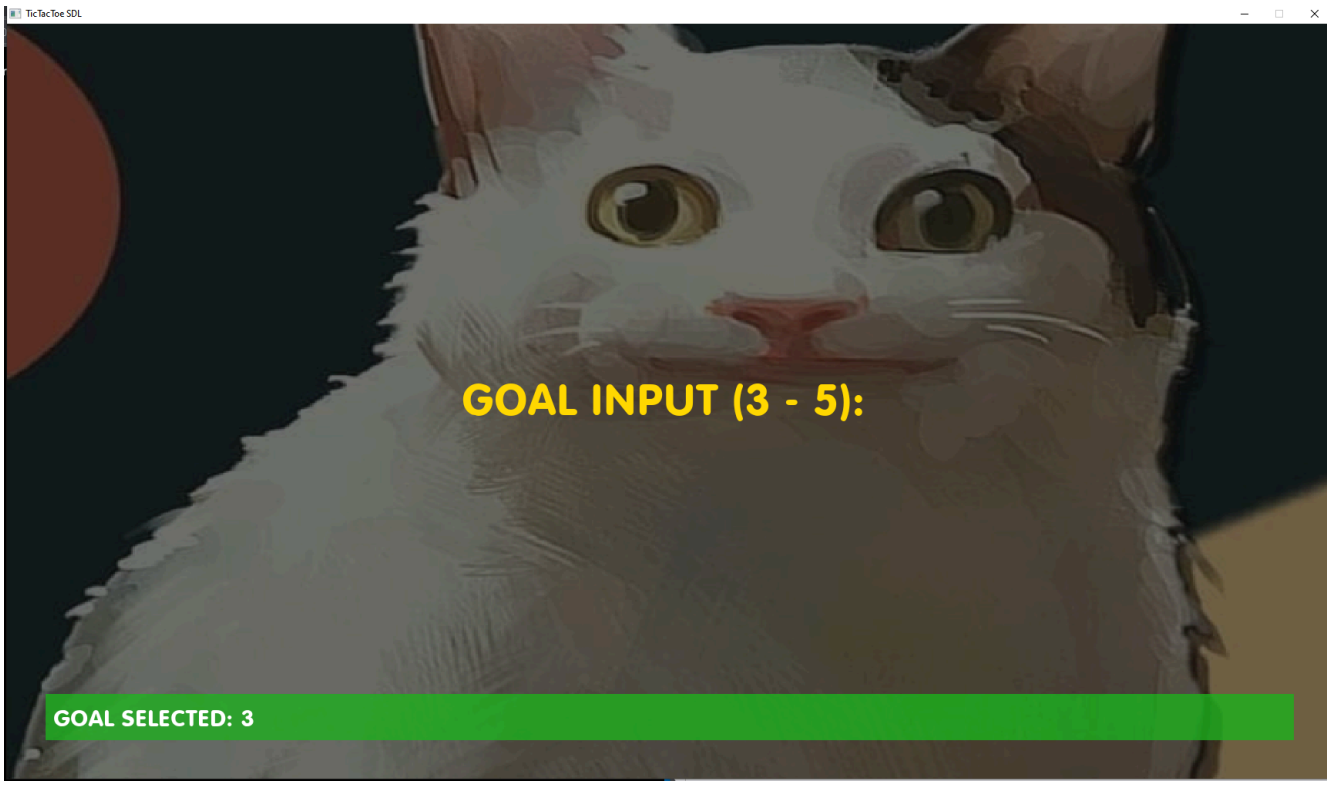

/d/dowload/level-2/build
0 1 2 3 4
0 |- - - - -
1 |- - O - X
2 |- - X - O
3 |- - - - -
4 |- - - - -
Bot (Player 1) is thinking...
Move placed at (3, 1).

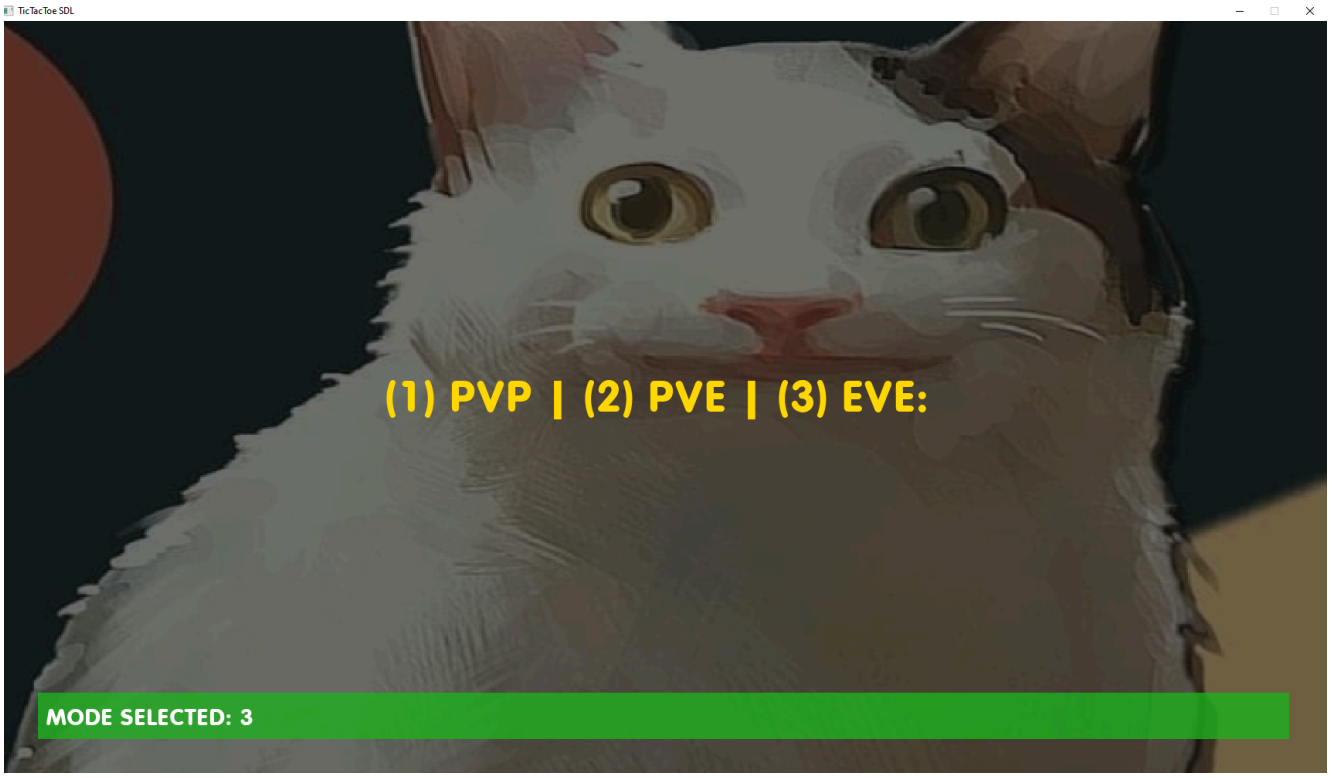


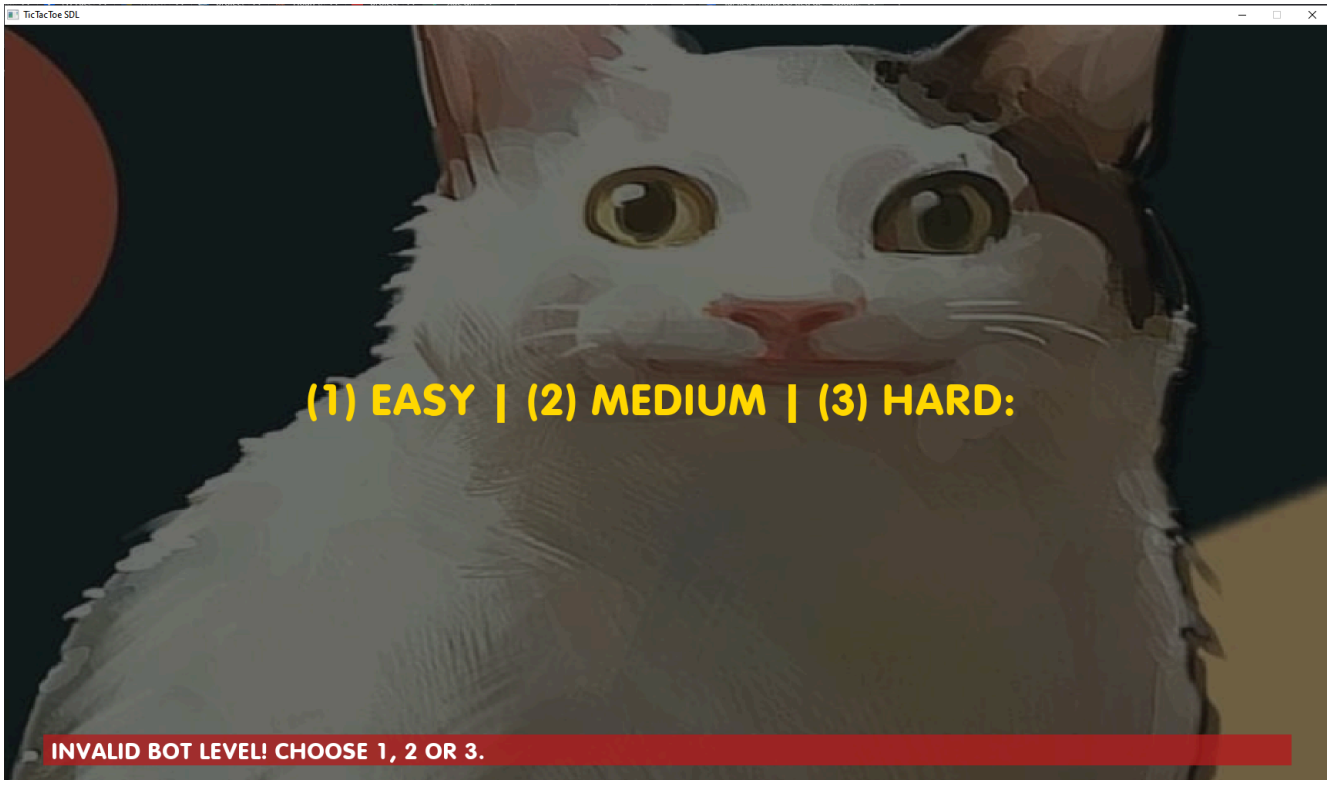
- SDL (Title, menu, displayboard, result):

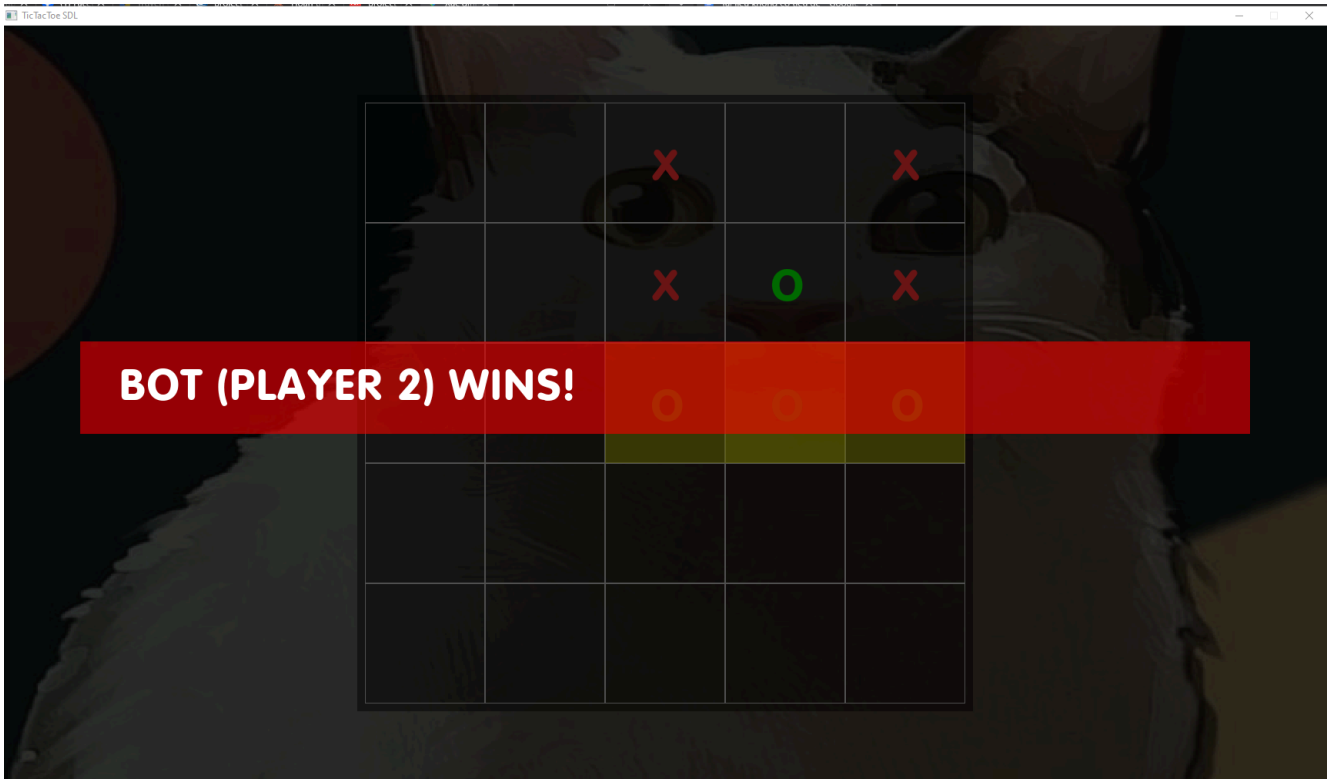



TicTacToe SDL
Press Alt+Z to u
Experience in-g
overlay
SIZE INPUT (NXN, 3<= N <=12):
SIZE SELECTED: 5


TicTacToe SDL
GOAL INPUT (3 - 5):
GOAL SELECTED: 3

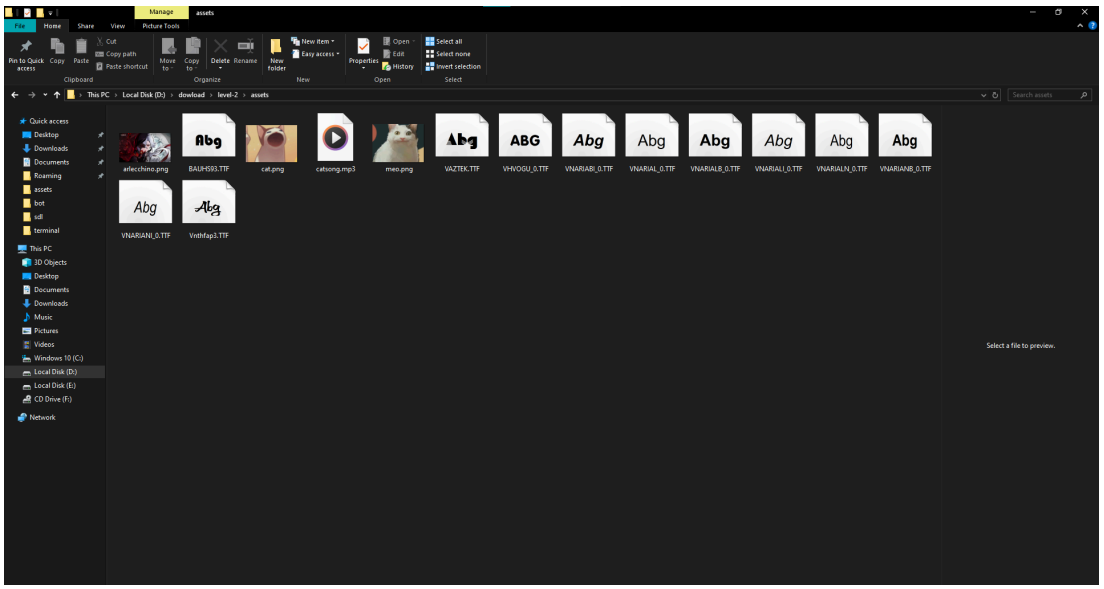

TicTacToe SDL
(1) PVP | (2) PVE | (3) EVE:
MODE SELECTED: 3


TicTacToe SDL
(1) EASY | (2) MEDIUM | (3) HARD:
INVALID BOT LEVEL! CHOOSE 1, 2 OR 3.



***) Tài nguyên:**

Đối với hình ảnh: tải ảnh dạng .png về assets và dùng làm ảnh nền (đã thấy ở trên);



- Đối với font chữ: vào ổ C → Windows → Fonts → chọn font mình thích (bài của em cả title lẫn chữ bình thường đều dùng cùng 1 font)

- Cài đặt trong Init():

```cpp
.d SDLRenderer::init(const RunConfig &config)

  // read config
  screenWidth = config.screenWidth;
  screenHeight = config.screenHeight;
  boardPadding = config.boardPadding;
  // int boardPadding = config.boardPadding;
  // init(...)

  SDL_Init(SDL_INIT_VIDEO);
  TTF_Init();
  IMG_Init(IMG_INIT_PNG);

  window = SDL_CreateWindow(
      "TicTacToe SDL",
      SDL_WINDOWPOS_CENTERED,
      SDL_WINDOWPOS_CENTERED,
      screenWidth,
      screenHeight,
      0);

  renderer = SDL_CreateRenderer(window, -1, SDL_RENDERER_ACCELERATED);

  SDL_SetRenderDrawBlendMode(renderer, SDL_BLENDMODE_BLEND);

  // load font
  // font = TTF_OpenFont("assets/font.ttf", font_size);
  font = TTF_OpenFont("D:/dowload/level-2/assets/VHVOGU_0.TTF", 32);
  fontTitle = TTF_OpenFont("D:/dowload/level-2/assets/VHVOGU_0.TTF", 60);
  SDL_Surface *bgSurface = IMG_Load("D:/dowload/level-2/assets/meo.png");
  if (bgSurface)
  {
      bgTexture = SDL_CreateTextureFromSurface(renderer, bgSurface);
      SDL_FreeSurface(bgSurface);
  }
```

*) Trích dẫn log:

log_graphic_flag.txt - Notepad

File Edit Format View Help

```
Tic-tac-toe Game (Version: 0.4.<NGUYENTHAOLY_25022931> by baluong.87)
--------------------------------------------------
[INFO] - [2026-05-23 00:39:46] Logger initialized!
[INFO] - [2026-05-23 00:39:46] SDL Renderer initialized!
[INFO] - [2026-05-23 00:39:46] SDL Interaction initialized!
[INFO] - [2026-05-23 00:39:46] Infra initialized!
[INFO] - [2026-05-23 00:39:46] Engine initializing . . .
[INFO] - [2026-05-23 00:39:46] Engine initialized!
[INFO] - [2026-05-23 00:39:46] [Engine] Starting game . . .
[INFO] - [2026-05-23 00:39:48] user input 'size' = 5
[INFO] - [2026-05-23 00:39:50] user input 'goal' = 3
[INFO] - [2026-05-23 00:39:52] user input 'game mode' = EVE
[INFO] - [2026-05-23 00:39:54] Board initialized!
[INFO] - [2026-05-23 00:39:54] [Engine] Game started!
[INFO] - [2026-05-23 00:39:54] [Engine] Playing game . . .
[INFO] - [2026-05-23 00:39:54] [Engine] starting turn #0
[INFO] - [2026-05-23 00:39:54] player 1 make move to (2, 2)
[INFO] - [2026-05-23 00:39:56] [Engine] starting turn #1
[INFO] - [2026-05-23 00:39:56] player 2 make move to (1, 2)
[INFO] - [2026-05-23 00:39:57] [Engine] starting turn #2
[INFO] - [2026-05-23 00:39:57] player 1 make move to (1, 4)
[INFO] - [2026-05-23 00:39:59] [Engine] starting turn #3
[INFO] - [2026-05-23 00:39:59] player 2 make move to (2, 4)
[INFO] - [2026-05-23 00:40:01] [Engine] starting turn #4
[INFO] - [2026-05-23 00:40:01] player 1 make move to (3, 1)
[INFO] - [2026-05-23 00:40:02] [Engine] starting turn #5
[INFO] - [2026-05-23 00:40:02] player 2 make move to (1, 3)
[INFO] - [2026-05-23 00:40:04] [Engine] starting turn #6
```

Ln 1, Col 1 | 100% | Windows (CRLF) | UTF-8

D. NHẬN XÉT VÀ TỔNG KẾT BÀI LÀM

1. **Phòng cách lập trình:**
   Điểm mạnh: Việc áp dụng lập trình hướng đối tượng giúp phân tách rõ ràng các thực thể trong trò chơi (như SDLRenderer chuyên lo hiển thị, SDLInteraction chuyên bắt sự kiện). Điều này giúp luồng trò chơi trở nên mạch lạc, dễ kiểm soát và không bị chồng chéo logic.

   Điểm yếu: Để làm một tính năng nhỏ, lập trình viên phải tạo rất nhiều file, định nghĩa cấu trúc lớp, hàm ảo phức tạp. Trong khi nếu viết theo kiểu tuyến tính thông thường thì chỉ cần vài dòng lệnh là xong.

   Bảo trì dự án lớn: Khi dự án trở nên phức tạp (thêm luật chơi mới, thêm chế độ nâng cao), cách tiếp cận này cực kỳ dễ bảo trì. Chúng ta có thể sửa đổi hoặc mở rộng tính năng của một lớp mà hoàn toàn không làm ảnh hưởng hay phá vỡ cấu trúc của các lớp khác.

   Sự phù hợp: Không phải dự án nào cũng phù hợp với OOP. Với các dự án siêu nhỏ, các script xử lý tự động hóa ngắn gọn hoặc các thuật toán thuần túy, lập trình thủ tục hoặc lập trình hàm sẽ mang lại tốc độ triển khai nhanh và hiệu năng tốt hơn.

2. **Về giao diện người dùng (GUI):**
   Điểm thú vị: Việc tự tay làm chủ thư viện SDL2 để tạo nên một bàn cờ Caro có màu sắc sinh động, có khả năng tự co giãn kích thước ma trận linh hoạt (cellsize = boardsize / gamesize) và có các hiệu ứng highlight nước đi vừa đánh đem lại trải nghiệm thị giác rất kích thích.

   Khó khăn gặp phải: Trở ngại lớn nhất là việc xử lý đồng bộ luồng và chuyển cảnh giao diện (ví dụ: việc chặn logic tại màn hình Title, bắt sự kiện phím ENTER lần 1 để chuyển sang màn hình nhập Size mà không làm treo hay lỗi hiển thị GUI).

   So sánh với Terminal: Bản thân em thích giao diện đồ họa (GUI) hơn. Mặc dù Terminal viết code rất nhanh và nhẹ, nhưng GUI vượt trội hoàn toàn về mặt trải nghiệm người dùng (UX). Nó cho phép tương tác chuột

trực quan, hiển thị bàn cờ kích thước lớn mà không lo bị vỡ khung chữ, và truyền tải trạng thái game một cách rõ ràng qua màu sắc.

3. **Về tổ chức mã nguồn:**
   Hiệu quả lâu dài: Việc tách mã nguồn thành nhiều file nhỏ kết hợp viết docstring/comment kỹ thuật là điều bắt buộc cho việc phát triển lâu dài. Nó giúp lập trình viên không bị lú code và dễ dàng định vị vị trí cần sửa đổi.
   So sánh tổ chức mã nguồn:

   Tập trung vào 1 file: Triển khai nhanh lúc đầu với dự án nhỏ, nhưng khi code lên tới hàng nghìn dòng sẽ cực kỳ hỗn loạn, khó đọc và dễ gây xung đột khi làm việc nhóm.

   Tách nhiều file: Code sạch sẽ, dễ tái sử dụng. Nhược điểm duy nhất là cấu trúc quản lý file ban đầu phức tạp và cần cấu hình build (CMake) cẩn thận.

   Khi nào lựa chọn: Nên chọn tổ chức nhiều file ngay khi dự án bắt đầu có từ 2 thành phần độc lập trở lên hoặc khi dự án được định hướng sẽ mở rộng quy mô trong tương lai.

4. **Về thiết kế chương trình và công cụ AI:**
   Vai trò của con người và AI: AI là một công cụ hỗ trợ hữu ích trong việc tăng tốc viết mã nguồn hoặc gợi ý các cú pháp thư viện đồ họa phức tạp. Tuy nhiên, tư duy thiết kế kiến trúc hệ thống và giải quyết bài toán thực tế vẫn hoàn toàn phụ thuộc vào con người. AI không thể thay thế con người ở khả năng hiểu sâu sắc ngữ cảnh hệ thống và đưa ra các quyết định thiết kế tổng thể.

   Vấn đề của AI hiện tại: Các công cụ AI hiện nay thường gặp hiện tượng hallucination, sinh ra các đoạn code mẫu chắp vá, thiếu đồng nhất về mặt kiến trúc tổng thể, hoặc dễ bỏ sót các lỗi logic tiềm ẩn về quản lý bộ nhớ.

5. **Bài học rút ra:**

   Khó khăn lớn nhất đã vượt qua: Đó là thành công nghiên cứu và code ra botLevel3 (phần mất nhiều thời gian nhất). Vì muốn tự mình nghĩ ra thuật toán riêng cho bot level 3 mà trong quá trình làm xảy ra nhiều sai sót,

chưa có sự tính toán kĩ lưỡng để các nước đi của bot level 3 bị che mắt dẫn đến thua bot level 2 liên tục. Sau đó để tiết kiệm thời gian, em đã ngồi tập trung nghiên cứu và học hỏi từ thuật toán minimax và thấy có lẽ minimax là cách tốt nhất hiện tại. Đây là vấn đề về lập trình.

E. GHI CHÚ SỬ DỤNG AI:

- Công cụ AI sử dụng: Claude

- Mục đích: debug, tìm hiểu về cách tạo botLevel3 và tìm nguồn học phù hợp về SDL. Cụ thể thì, trong quá trình làm bài rất dễ mắc error, ban đầu khó tìm được vị trí lỗi nên vẫn cần đến AI chỉ cho nhưng chỉ dùng 2 3 lần là bắt đầu quen với các hàm nên không cần tới AI để tìm lỗi nữa; dùng AI nhiều nhất là đối với bot level 3 để lên ý tưởng tư duy thuật toán cho bot, em đã cho bot level 3 đánh với bot level 2 khoảng 50 lần, trong đó thua tới 48 - 49 trận (em không dùng hàm random để chơi mà dùng std::uniform_int_distribution để dễ debug và theo dõi nước đi hơn), trong quá trình đó em gần như không đụng vào AI nhưng cuối cùng em đã đổi ý và học cách sử dụng minimax theo hướng dẫn của claude. Đối với SDL ban đầu khá lạ lẫm và không biết dùng như nào, nhưng có sự hỗ trợ của AI gợi ý và giải thích nên dần dần đã thích nghi.

- Mức độ đóng góp: 30% (sử dụng chủ yếu về cách dùng SDL, botLevel3 và debug)

F. NGUỒN THAM KHẢO:

https://lazyfoo.net/tutorials/SDL/index.php
Algorithms Explained – minimax and alpha-beta pruning